Global scoreboarding实现方案

基于timeout机制的speculative precharge策略

依然是只有行命中簇的末尾项需要考虑timeout计时,目前已经实现了。

1. 当行命中簇的末尾项发射之后，开始timeout计时：
   a. timeout计时到时之后如果该bank没有新的请求来到，就发送precharge命令，关闭这个行
   b. 在timeout计时结束之前，如果有了新的行命中请求来到，则重置该timeout值
   c. 在timeout计时结束之前，有了新的行冲突请求来到，则马上发送precharge命令关闭现在的行。

timeout更新

```
Repeat Every N Requests:
    T = currTimeout
    # Compute hit and conflict increments from scoreboard
    for t in {timeout windows in scoreboard}:
        hitsIncr[t] = hits[t] - hits[T]
        conflictsIncr[t] = conflicts[t] - conflicts[T]
        hits[t] = conflicts[t] = 0

    # Pick the timeout that maximizes hits,
    # and minimizes conflicts
    nextT = argmax(hitsIncr[] - conflictsIncr[])

    # If variation is not substantial, do not change timeout
    if max(hitsIncr[] - conflictsIncr[]) <
        (1 + variationThreshold) * min(hitsIncr[] -
            conflictsIncr[]):
        nextT = T
    # nextT represents the timeout for the next N requests
```

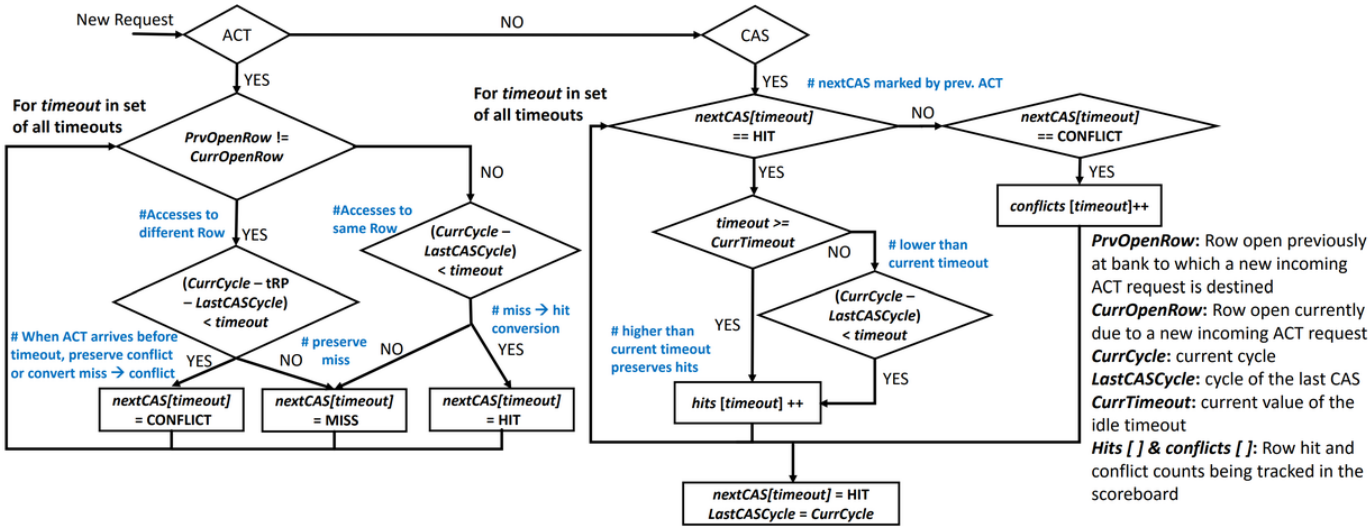
timeout值是按照per bank的方式设定的

预设的几个timeout window，**timeout window的具体的预设值?**

对于这些timeout window，计算hitsIncr和conflictsIncr，然后选择最大化 `hitsIncr-conflictsIncr` 的timeout window

如果波动足够明显，才更新timeout值

hits和conflicts向量的计算



这张 Figure 6 流程图展示了**Global Scoreboarding** 机制的核心逻辑：即“Scoreboard Update”（记分板更新算法）。

它的核心目的是：**对于每一个到达的内存请求，通过“假设”不同的 Timeout 窗口（例如 50, 100, 200 cycles），来推演该请求在这些假设下会产生 Row Hit（行命中）、Row Miss（行缺失）还是 Row Conflict（行冲突），并以此更新记分板**。

为了让你更容易理解，我们可以把这个流程分为两个阶段：**ACT（激活）阶段**和 **CAS（读写）阶段**。算法对每个设定的 `timeout` 值（如 50, 100, ...）都会并行跑一遍这个流程。

---

1. 核心思想：通过“如果……会怎样”来推演

这个算法在问一个问题：**“如果当前的 idle timeout 是 X cycles，刚才这个请求会发生什么？”**

- **ACT 处理分支（左侧）：** 负责**定性**。当一个 ACT 命令到来时，判断它对于特定的 timeout 值来说，是会造成冲突（Conflict），还是本该命中却因为 timeout 太短变成了 Miss。
- **CAS 处理分支（右侧）：** 负责**记分**。当 CAS 命令真正执行数据传输时，根据之前的定性，更新 `hits[]` 和 `conflicts[]` 计数器。

---

2. 详细流程解析

第一阶段：ACT (Activate) 命令处理（流程图左半部分）

当一个新的请求是 `ACT` 命令时，意味着内存控制器决定打开某一行。算法需要判断这个行为在不同 `timeout` 假设下的性质。

- **判断是访问同一行还是不同行？**（`PrvOpenRow != CurrOpenRow`）
  - **情况 A：访问不同行（Different Row） -> 潜在的 Row Conflict**
    - **逻辑：** 如果我们在上一行空闲后等待了 `timeout` 这么久才关闭它，而新请求来得太快，就会导致我们还没来得及预充电（Precharge），必须强制先 Precharge 再 Activate，这就构成了 Row Conflict。
    - **公式判断：** `(CurrCycle - tRP - LastCASCycle) < timeout` 。
      - 这里 `CurrCycle` 是当前时间，`LastCASCycle` 是上一行最后一次读写的时间。
      - 意思是：如果“上一行读写完”到“现在需要开新行”的时间间隔太短，短到还不够跑完假设的 `timeout` 窗口，那么在这个假设下，行还开着，必须强行关闭，所以标记为 **CONFLICT** 。
  - **情况 B：访问同一行（Same Row） -> 潜在的 Miss 转 Hit**
    - **逻辑：** 既然现在发了 ACT，说明当前系统认为它是 Row Miss（行已关闭）。但如果假设的 `timeout` 足够长，也许行还开着，这就变成了一个 **HIT**。
    - **公式判断：** `(CurrCycle - LastCASCycle) < timeout` 。
      - 意思是：如果上次读写完到现在的时间间隔，小于假设的 `timeout` ，说明在那个假设世界里，行还没关呢！
      - **结果：** 将状态标记为 **HIT**（意味着如果 timeout 设为这个值，刚才那个 Miss 其实能变成 Hit）。否则标记为 **MISS**。

第二阶段：CAS (Column Access) 命令处理（流程图右半部分）

当 `CAS` 命令到来时，通常紧跟在 ACT 之后（或者是对已打开行的访问）。这里负责根据刚才 ACT 阶段设定的状态（nextCAS）来更新计数器。

- **检查预设状态 (`nextCAS[timeout]`)**
  - **如果是 CONFLICT：**
    - 直接让 `conflicts[timeout]` 计数器加 1 。这表示在这个 timeout 设置下，不仅没带来好处，还增加了冲突惩罚。
  - **如果是 HIT：**
    - 这意味着在假设的 timeout 下，这一行是开着的。

- **逻辑分支：**
  1. **timeout >= CurrTimeout（当前实际使用的超时）：** 如果假设值比当前实际值大，说明能保持更多 Hit，直接记为有效 Hit。
  2. **timeout < CurrTimeout：** 如果假设值比当前实际值小（更激进的关闭策略），我们需要再次确认：在这个较短的 timeout 下，请求间隔是否真的足够短？
     - `(CurrCycle - LastCASCycle) < timeout` 。
     - 如果是，说明即使 timeout 设得这么短，也能 Hit，于是 `hits[timeout]` 加 1 。
- **收尾工作：**
  - 处理完 CAS 后，把 `nextCAS` 重置为 **HIT**（因为行现在是开着的，后续紧接着来的 CAS 默认都是 Hit）。
  - 更新 `LastCASCycle` 为当前时间。

---

3. 总结global scoreboarding的流程图的意义

这个图展示了一个并行的影子模拟过程。

系统在实际运行一个固定的 timeout（比如 50）的同时，会在后台默默计算：

- “如果 timeout 是 100，刚才那个 Miss 会不会变成 Hit？” -> (如果是，记分板 100 分栏 +1 Hit)
- “如果 timeout 是 200，刚才那个操作会不会导致 Row Conflict？” -> (如果是，记分板 200 分栏 +1 Conflict)

通过 该算法流程图的逻辑，经过一段时间（比如 N 个请求）后，Memory Controller 就可以查看记分板，选出那个**Hit 数增加最多且 Conflict 增加不明显**的 timeout 值，作为下一个阶段的实际策略。

row exclusion

row exclusion store是 per channel的，所有bank共享。

detection

记录每个bank发射的前一项请求访问的row，并且记录它是不是**因为timeout**被关闭掉了。

如果下一个访存请求访问的是一个刚刚被关闭掉的row，说明这个row是一个长寿行，则把这个地址（rank，bg，bank，row）放到一个64项的缓存里。

execution

当一个alive row即将timeout的时候，如果这个row可以在row exclusion store被检索到，则不要立即关闭这个row，直到一个访问别的row的请求**到达**队列中，才关闭这个row。这么看来，这种机制其实主要是在请求非常稀疏的场景下起作用。

row exclusion store的替换策略

当row exclusion store满了之后，优先替换引发最近的row conflict的地址。

可以实现为一个类似FIFO的结构，正常的时候就按FIFO形式进行组织，如果某个row因为命中了row exclusion store的某项地址获得了更长的存活时间（本来应该因为timeout关闭），然后一个访问别的row的请求**到达**队列中，造成了行冲突（本来timeout到时间，早就该关闭了），就表示row exclusion store的某项地址引发了row conflict，需要把这一项地址移到队头，当row exclusion store满时，优先替换之。